

Introduction to Python

EMATM0048- Teaching Session 1B

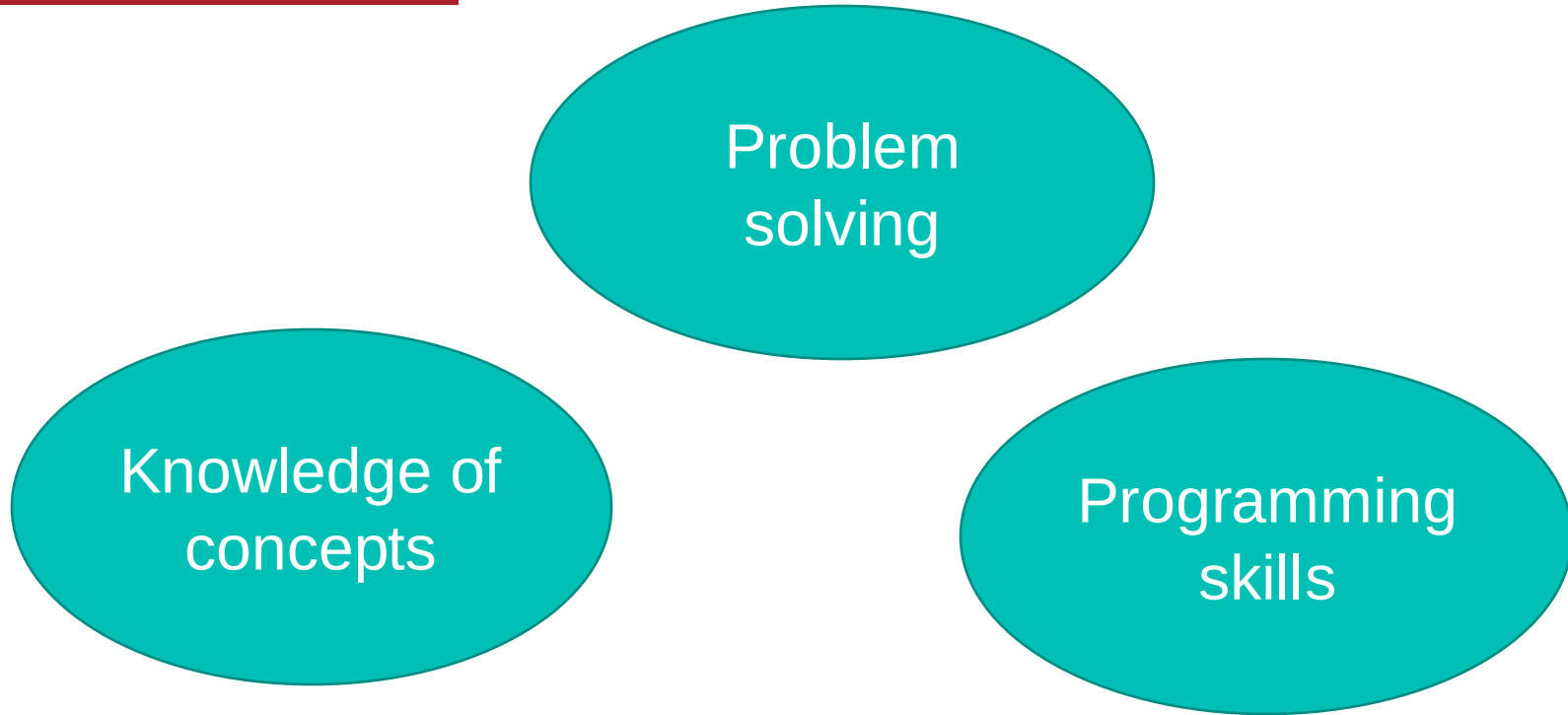


Before we start!

Is this unit about Python?

- Although we use Python a lot, this unit is not about Python.
- What is much more important, however, is that you will learn something about how to write programs that solve problems.
- This skill can be transferred to any programming language.

Week1

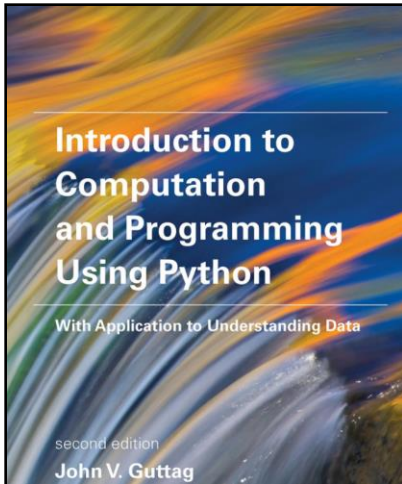


Session's contents

- Basic elements of Python
- Python modes
- Objects in python
- Expressions
- Input and output operators

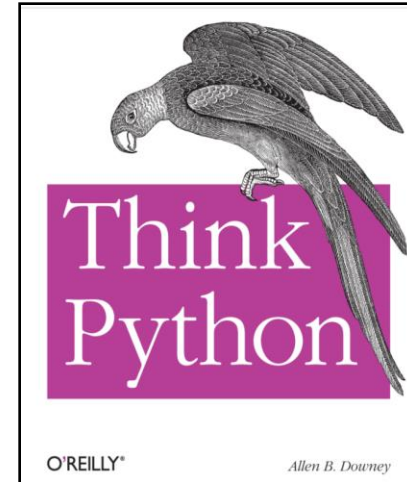
Readings

Chapter 2 (2.1) Introduction to Python



bristol.ac.uk

Chapter 2: Variables, expressions and statements



Basic elements of Python

- A command, often called a **statement**, instructs the interpreter to do something.

```
print ("Hello World!")
```

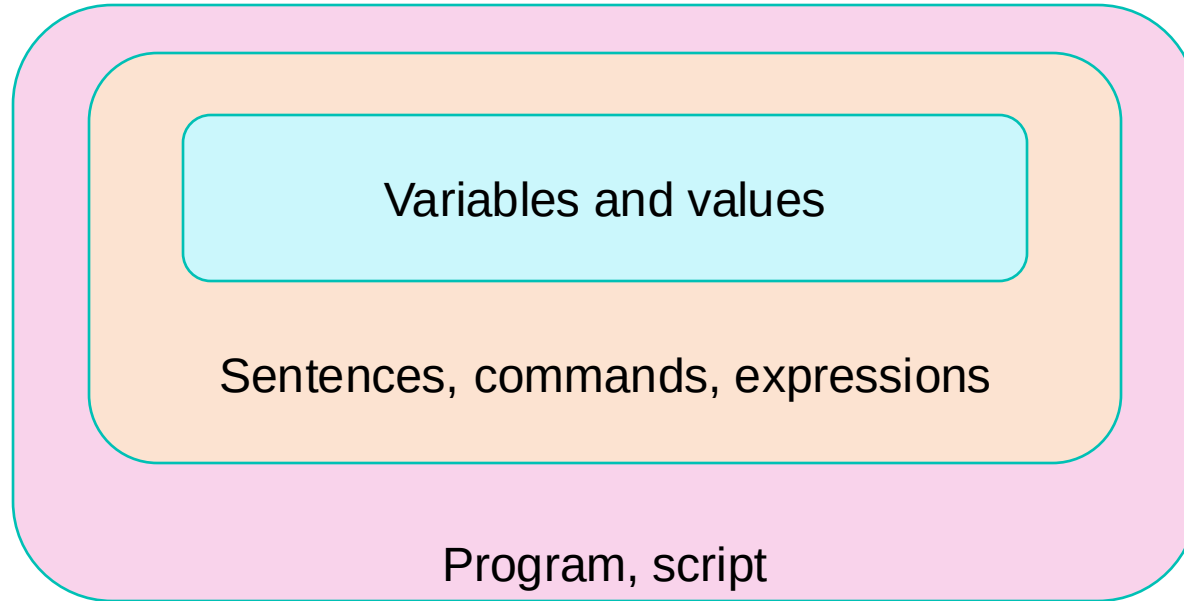
- A Python **program**, sometimes called a **script**, is a sequence of definitions and commands.

Basic elements of Python

```
print ("First line")  
print ("Second line")  
print ("This looks like a script")
```

- These definitions are evaluated and the commands are executed by the Python **interpreter** in something called the **shell**.

Elements of Python



Session's contents

- The basic elements of Python
- Python modes
- Objects in python
- Expressions
- Input and output operators

Python modes



Try your shell

Shell /interactive mode:

- The symbol `>>>` is a shell prompt indicating that the interpreter is expecting the user to type some Python code into the shell.
- The line below the line with the prompt is produced when the interpreter evaluates the Python code entered at the prompt.
- Typing programs directly into the shell is highly inconvenient. Most programmers prefer to use some sort of text editor that is part of an **integrated development environment (IDE)**.

Python modes

Script mode

- IDLE provides a text editor with syntax highlighting, autocompletion, and smart indentation.
- A shell with syntax highlighting.
- An integrated debugger

Interactive vs. script

- Interactive
 - You type directly to Python one line at a time and it responds
- Script
 - You enter a sequence of statements (lines) into a file using a text editor and tell Python to execute the statements in the file, line by line.

Test in Spyder!

Elements of Python

Variables and values

Session's contents

- The basic elements of Python
- Python modes
- **Objects in python**
- Expressions
- Input and output operators

Objects in Python

- **Objects** are the core things that Python programs manipulate.
- Every object has a **type** that defines the kinds of things that programs can do with objects of that type.
 - Alpaca is an animal so can walk, run, can't speak!
- Objects are either **scalar** or **non-scalar**
 - **Scalar**: cannot be subdivided (atoms)
 - **Non-scalar**: have internal structure

Scalar Objects

- int - represent **integers**, ex. 5
 - float –represent **real numbers**, ex. 3.27
 - bool –represent **Boolean** values True and False
 - NoneType – Special and has one value, None
-
- type() built-in function in python shows the type of an object

```
>>type(5)  
>>type(5.3)  
>>type(None)
```

Try in your
shell

Type conversions (cast)

- Can convert object of one type to another

float(3) converts integer 3 to float 3.0

int(3.9) truncates float 3.9 to integer 3

- Can this work all the time?

Try convert string to integer

Try convert float to string

Commenting in Python

```
# A single-line comment is denoted with the hash symbol.
```

```
"""
```

```
Multi-line comments
```

```
Lie between quotation marks
```

```
"""
```

Binding variables and values

- Equal sign is an assignment of a value to a variable name
- value stored in computer memory
- An assignment binds name to value
- Retrieve value associated with name or variable by invoking the name, by typing the variable name

```
>> x= 12.5  
>>x  
12.5
```

Try in your
shell

Why naming?

- To reuse names instead of values
- Easier to change code later

```
>>length= 12  
>>width= 4  
>>area= width*length
```

Try in your
shell

A Type Dream

- Variables in Python are dynamically typed. They take on the type of the object they are representing

Try in your
shell

```
int x = 5; # C/C++/Java
```

```
>> x = 5 # Python - no type! No semicolon! What just happened?
```

A Type Dream

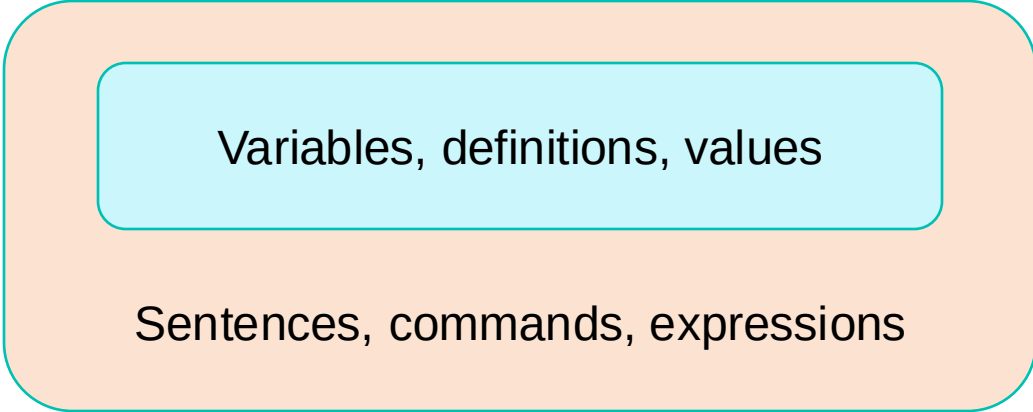
```
>> x= 1
>> type(x)
int

>> x= 'Alpaca'
>> type(x)
str
```

Session's contents

- The basic elements of Python
- Python modes
- Objects in python
- Expressions
- Input and output operators

Elements of Python



A diagram consisting of two nested rounded rectangles. The outer rectangle is light orange and contains two lines of text. The inner rectangle is light blue and contains one line of text.

Variables, definitions, values

Sentences, commands, expressions

Expressions

- Python combines **objects** and **operators** to form expressions
- We learnt about objects. What are operators?
- An expression has a value, which has a type
- Syntax for a simple expression
 <object> <operator> <object>

What are the mathematical operators?

- Sum: $x + y$
- Difference: $x - y$
- Product: $x * y$
- Division: x / y
- Modulo: $x \% y$
- Exponential: $x ** y$

Is the output integer or float?

Mathematical operators

- The floor division operator, `//`, divides two numbers and rounds down to an integer.
- Example:
 - suppose the run time of a movie is 105 minutes. You might want to know how long that is in hours. Conventional division returns a floating-point number: 1.75
 - But we don't normally write hours with decimal points. Floor division returns the integer number of hours, rounding down
 - To get the remainder, you could use modulo

Operators examples

```
>> minutes = 105  
>> minutes / 60  
1.75  
>> hours = minutes // 60  
>> hours  
1  
>> remainder = minutes % 60
```

What is the value of remainder?

- a. 65
- b. 45
- c. 75
- d. 1

Can you use modulo operator to check whether a number is odd or even?

More mathematical operators

- Appending an = to any of the prior operations allows for assignment based on the prior value of the variable

$x += 5$		$x = x + 5$
$x -= 3$		$x = x - 3$
??		$x = x * 2$
??		$x = x / 2$

If $x=4$, what is the value of $x^{**}=3$?

- a. 12
- b. 96
- c. 27
- d. 64

Comparison operators

- Comparison operators returns Boolean (TRUE or FALSE)
- The == operator is used to test whether two expressions evaluate to the same value.
- The != operator is used to test whether two expressions evaluate to different values.
- More comparison operators: < , > , <=, >=

```
>> 4 == 5  
False  
>> 4 != 5  
True
```

Try in your
shell

What is the difference between `==` and `=`

- Think about what is the output of each.
- If `x= 3` and `y=5`, which statement changes the value of `x`?

➤ `x= y`

➤ `x==y`

- a. Both changes the value of `x`
- b. `x=y` changes the value of `x`
- c. `x==y` changes the value of `x`
- d. Neither

Logic operators

A	B	A and B	A or B
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

```
>> surfing = True
>> work = False
>> print ( surfing and work)
>> print (surfing or work)
```

Programming vs. Maths

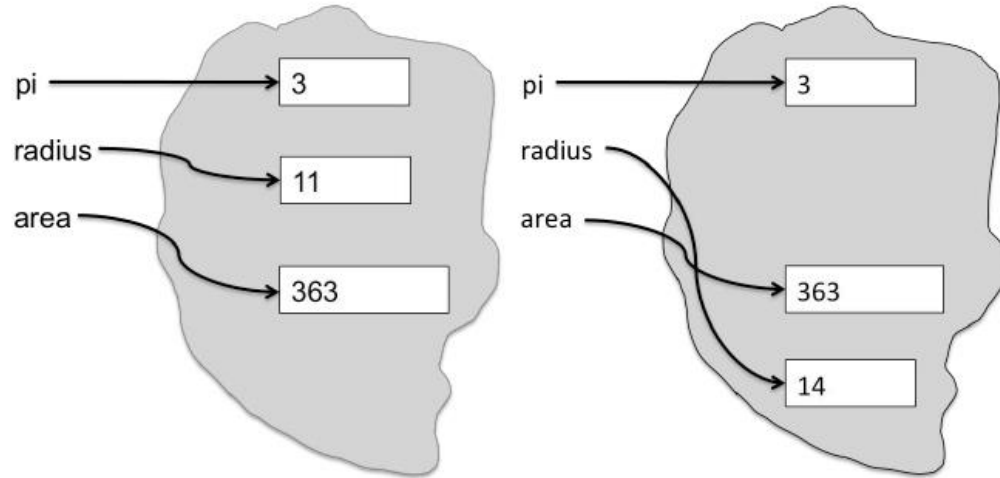
- in programming, you do not “solve for x”

```
>> pi= 3  
>> radius=11  
>> area= pie* (radius **2)  
>> radius= raduis+3 #can also be written as radius+=3
```

Try in your
shell

Changing bindings

- Can **re-bind** variable names using new assignment statements
- previous value may still be stored in memory but handle lost
 - value for area does not change until you tell the computer to do the calculation again



ON NAMING!



A variable is just a name?

- Is there a difference?
- Why names?
- Not wrong, but best practices for readability
- Commenting to increase readability *#calculating area of a circle....*

```
>> pi= 3.14159  
>> diameter=11.2  
>> area= pie* (diameter **2)
```

```
>> a= 3.14159  
>> b=11.2  
>> c= a* (b**2)
```

On naming!

- Variable names can contain uppercase and lowercase letters, digits and the special character `_`.
- Variable names **cannot start with a digit**
- Variable names are **case-sensitive** e.g., `alpaca` and `Alpaca` are different names.
- There are a small number of **reserved words** (sometimes called keywords) in Python that have built-in meanings and cannot be used as variable names

Reserved Words

You cannot use reserved words as variable names / identifiers

<code>False</code>	<code>class</code>	<code>return</code>	<code>is</code>	<code>finally</code>
<code>None</code>	<code>if</code>	<code>for</code>	<code>lambda</code>	<code>continue</code>
<code>True</code>	<code>def</code>	<code>from</code>	<code>while</code>	<code>nonlocal</code>
<code>And</code>	<code>del</code>	<code>global</code>	<code>not</code>	<code>with</code>
<code>as</code>	<code>elif</code>	<code>try</code>	<code>or</code>	<code>yield</code>
<code>assert</code>	<code>else</code>	<code>import</code>	<code>pass</code>	<code>break</code>
<code>except</code>	<code>in</code>	<code>raise</code>	<code>print</code>	<code>input</code>

Session's contents

- The basic elements of Python
- Python modes
- Objects in python
- Expressions
- Input and output operators

Input/Output: `print` (" ")

- **Print:** used to **output** stuff to console.

```
>> x=10  
>> print (x)
```

```
>> x_str= str(x)  
>> print (" x is a number", x, ".")  
>> print (" x is a string "+x_str+ ".")
```

Input/Output: `input (" ")`

Try in your
shell

- Takes a string as an argument and displays it as a prompt in the shell.
- Then waits for the user to type something, followed by hitting the enter key.
- Binds that **value** to a **variable**

```
>> animal= input("What is your favourite animal?")
What is your favourite animal? ... Alpaca
>> print("Do you really like "+ animal + "?")
Do you really like Alpaca?
```

Input/Output: `input (" ")`

- Input is always a string, must be cast to other types if needed.

Cast and try again

```
>> number= input("enter a number.. ")  
>> type(number)  
>> print (5*number)
```

Takeaways

- Python consists of variables/definitions >> commands >> program
- Every variable in Python is an OBJECT
- Expressions are bindings objects and operators
- Scripting and interactive are used for different purposes in python
- A variable is just a name...Not really!

Next session:

- String type
- Branching and conditionals
- Python Indentation
- Iteration and loops